

CS206 — Electronic Commerce

Database Transactions

Complete Study Notes

Topics Covered

ACID Properties • Concurrency Problems • Isolation Levels Schedules • Recoverability •
Serializability • Precedence Graphs

1. What is a DB Transaction?

A **Transaction** is a process involving one or more database queries and/or modifications, treated as a single logical unit of work.

Why do we need transactions?

Database systems are normally accessed by **many users or processes at the same time**. Without transactions, concurrent access causes data corruption.

ATM Example: You and your friend each withdraw Rs10,000 from different ATMs at the same time. The DBMS must ensure both deductions are recorded and neither is lost.

2. ACID Properties

Fund Transfer Example — Transfer \$50 from Account A to Account B:

Step	Operation
1	read(A)
2	$A := A - 50$
3	write(A)
4	read(B)
5	$B := B + 50$
6	write(B)

Property	Meaning	Fund Transfer Example
Atomic	Whole transaction or none is done	If crash after step 3 (write A), undo A's deduction — B never gets updated
Consistent	DB constraints always preserved	Sum of A+B is the same before and after: $\$500 + \$200 = \$450 + \$250 = \$700$
Isolated	Appears as if only one process runs at a time	No other transaction sees A deducted but B not yet updated
Durable	Effects survive crashes after commit	Once 'transfer complete' shown, data persists even if server crashes

3. T-SQL Transaction Modes

Autocommit: Every single SQL statement is automatically its own transaction.

Explicit Transaction syntax:

```
BEGIN TRANSACTION UPDATE Department SET Mgr_ssn = 123456789 WHERE DNumber = 1
UPDATE Department SET Mgr_start_date = '1981-06-19' WHERE Dnumber = 1 COMMIT TRAN
-- or ROLLBACK TRAN
```

Both UPDATES are wrapped together — either both succeed or neither does.

4. Concurrency Problems

These problems occur when isolation is NOT properly enforced.

4a. Dirty Read

Reading a value that was written by an **uncommitted** transaction. If that transaction rolls back, you acted on data that never officially existed.

TRANSACTION 1	TRANSACTION 2
Update Quantity to 5	
	Read Quantity as 5 ← DIRTY READ
Billing Customer	
Insufficient funds Rolled back Reverts Quantity to 10	

T2 read Quantity=5, but T1 rolled back. The real value was always 10. T2 acted on a lie.

4b. Non-Repeatable Read

A transaction reads the **same row twice**, but another transaction modifies it in between — returning two different values.

Alice (T1)	Database	Bob (T2)
BEGIN TRANSACTION	post: id=1, title='Transactions'	
		BEGIN TRANSACTION
	SELECT * FROM post WHERE id=1	
		Sees: title='Transactions'
UPDATE post SET title='ACID' WHERE id=1	post: id=1, title='ACID'	
COMMIT		
	SELECT * FROM post WHERE id=1	
		Now sees: title='ACID' ← DIFFERENT!

Bob read the same row twice and got two different values — Non-Repeatable Read!

4c. Phantom Read

A transaction runs the **same query twice** with a WHERE condition, but **new rows** inserted by another transaction appear in the second read — like a ghost row.

Alice (T1)	Bob (T2)
BEGIN TRANSACTION	
INSERT INTO post_comment(id, post_id) VALUES(4, 1)	BEGIN TRANSACTION
	SELECT * FROM post_comment WHERE post_id=1 → Sees 3 rows (id 1,2,3)
COMMIT	
	SELECT * FROM post_comment WHERE post_id=1 → Now sees 4 rows! ← PHANTOM

Row id=4 materialized out of nowhere in Bob's second query — that's the Phantom!

5. Isolation Levels in T-SQL

Level	SQL Command	Prevents
1. Read Uncommitted	SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED	Nothing – allows dirty reads
2. Read Committed	SET TRANSACTION ISOLATION LEVEL READ COMMITTED	Dirty reads
3. Repeatable Read	SET TRANSACTION ISOLATION LEVEL REPEATABLE READ	Dirty + Non-repeatable reads
4. Serializable (default)	SET TRANSACTION ISOLATION LEVEL SERIALIZABLE	All three problems

Higher isolation level = fewer problems, but lower concurrency and performance.

6. Schedules

A **schedule** is an ordering of operations from multiple concurrent transactions.

Serial Schedule

All operations of T1 run to completion, then all of T2. **Always correct** but slow — no parallelism.

Example: `r1(X) w1(X) r1(Y) w1(Y) | r2(X) w2(X)` – T1 fully finishes before T2 starts.

Non-Serial Schedule

Operations from different transactions are **interleaved**. Faster, but may give wrong results.

Example: `r1(X) r2(X) w1(X) r1(Y) w2(X) w1(Y)` – T1 and T2 interleaved.

Serializable Schedule

A non-serial schedule that produces the **same result** as some serial schedule. It's interleaved but still correct.

Serializable = Correct result, even though interleaved.

7. The Food Court Example (Why We Need Transactions)

Setup:

Table: Sells(shop, item, price). Ahmad sells Pizza (Rs200) and Sprite (Rs50).

Tania queries MAX and MIN price of Ahmad's shop at the same time Ahmad changes his menu.

Tania's Queries	Ahmad's Steps
(max) SELECT MAX(price) FROM Sells WHERE shop='Ahmad\'s shop'	(del) DELETE FROM Sells WHERE shop='Ahmad\'s shop'
(min) SELECT MIN(price) FROM Sells WHERE shop='Ahmad\'s shop'	(ins) INSERT INTO Sells VALUES('Ahmad\'s shop', 'Biryani', 250.00)

Bad Interleaving: (max)(del)(ins)(min)

Step	Ahmad's Prices	Result
(max)	{50, 200}	MAX = 200
(del)	{}	—
(ins)	{250}	—
(min)	{250}	MIN = 250

Tania sees MAX=200, MIN=250 — MAX < MIN! Mathematically impossible. The data is WRONG.

Fix: Group Tania's queries into one transaction. She sees Ahmad's prices at one fixed snapshot.

8. Recoverability of Schedules

Type	Rule	Protects Against
Non-Recoverable	T2 commits BEFORE T1 (after reading T1's data)	Nothing — cannot undo T2
Recoverable	T2 commits AFTER T1 (if T2 read T1's data)	Committed txns never need rollback
Cascadeless	T2 only reads COMMITTED data (no dirty reads)	Cascading rollbacks
Strict	T2 cannot read OR write X until last writer commits	All recovery problems

8a. Recoverable vs Non-Recoverable — Example

LEFT — Recoverable ■ T2 commits AFTER T1

RIGHT — NOT Recoverable ■ T2 commits BEFORE
T1 aborts

T1	T2	T1	T2
r(X)		r(X)	
w(X)		w(X) ← T1 writes X	
	r(X) ← reads T1's data		r(X) ← DIRTY READ
r(Y)		r(Y)	
	w(X)		w(X)
	commit		commit ← BEFORE T1!
w(Y)		abort	
commit			

8b. Cascading Rollback

If T1 fails and T2 read T1's data, T2 must also roll back. If T3 read T2's data, T3 must also roll back — a chain reaction of wasted work.

T1	T2	T3
r(X)		
r(Y)		
w(X)		
	r(X) ← reads T1	
	w(X)	
		r(X) ← reads T2
ABORT ■		

	must ABORT ■	
		must ABORT ■

8c. Cascadeless Schedule Fix

$r_2(X)$ must be **postponed until after T1 has committed (or aborted)**. This delays T2 but guarantees no cascading rollback.

Bad: $w_1(X) \rightarrow r_2(X)$ T2 reads before T1 commits – dirty read!

Good: $w_1(X) \rightarrow c_1 \rightarrow r_2(X)$ T2 only reads after T1 commits – safe!

Hierarchy: Strict $\subset$ Cascadeless $\subset$ Recoverable Each level is stricter than the one before.

9. Conflict Serializability

What is a Conflict?

Two operations CONFLICT if: 1) They belong to DIFFERENT transactions 2) They access the SAME data item 3) At least ONE is a WRITE

Pair	Conflict?	Type
r1(X) and w2(X)	YES ■	Read-Write
w1(X) and r2(X)	YES ■	Write-Read
w1(X) and w2(X)	YES ■	Write-Write
r1(X) and r2(X)	NO ■	Two reads – never conflict
r1(X) and w1(X)	NO ■	Same transaction – not a conflict

How to Test Serializability — Precedence Graph Method

Step	What to Do
1	Draw one NODE for each transaction (T1, T2, T3...)
2	Scan the schedule left-to-right for every pair of CONFLICTING operations
3	For each conflict: draw arrow $T_i \rightarrow T_j$ (T_i 's operation comes FIRST in the schedule)
4	Check the completed graph for CYCLES
5a	No cycle $\rightarrow$ SERIALIZABLE ■ (find serial order by topological sort)
5b	Cycle exists $\rightarrow$ NOT SERIALIZABLE ■

Worked Example — Schedule E (3 Transactions)

Transaction T1	Transaction T2	Transaction T3
	read_item(Z)	
	read_item(Y)	
	write_item(Y)	
		read_item(Y)
read_item(X)		read_item(Z)
write_item(X)		
		write_item(Y)
	read_item(X)	write_item(Z)

read_item(Y)		
write_item(Y)	write_item(X)	

Building the Precedence Graph step by step:

Conflict Found	Edge Added	Reason
T2 w(Y) before T3 r(Y)	T2 → T3 (Y,Z)	T2 writes Y first, T3 reads Y later
T2 w(Y) before T1 r(Y)	T2 → T1 (Y)	T2 writes Y first, T1 reads Y later
T3 w(Y) before T1 w(Y)	T3 → T1 (Y)	T3 writes Y first, T1 writes Y later
T1 w(X) before T2 r(X)	T1 → T2 (X)	T1 writes X first, T2 reads X later
T1 w(X) before T2 w(X)	T1 → T2 (X,Y)	T1 writes X first, T2 writes X later

Final graph has edges: T1→T2, T2→T1, T2→T3, T3→T1 Cycle detected: T1→T2→T1 and T1→T2→T3→T1 CONCLUSION: NOT conflict serializable — no equivalent serial schedule exists. ■

10. Quick Reference — Everything at a Glance

The 3 Concurrency Problems

Problem	What Happens	Prevented by Isolation Level
Dirty Read	Reading uncommitted data that may be rolled back	Read Committed (level 2+)
Non-Repeatable Read	Same row read twice gives different values	Repeatable Read (level 3+)
Phantom Read	Same query returns new rows the second time	Serializable (level 4)

The 4 Schedule Types (Recoverability)

Type	Rule	Protects Against
Non-Recoverable	T2 commits BEFORE T1 (after reading T1's data)	Nothing — cannot undo T2
Recoverable	T2 commits AFTER T1 (if T2 read T1's data)	Committed txns never need rollback
Cascadeless	T2 only reads COMMITTED data (no dirty reads)	Cascading rollbacks
Strict	T2 cannot read OR write X until last writer commits	All recovery problems

Serializability Check — One-Page Summary

■ Is it Serializable?	Precedence Graph Method
Step 1	Find all CONFLICTING operation pairs (different T, same item, at least one write)
Step 2	Draw $T_i \rightarrow T_j$ for each conflict where T_i 's op comes first
Step 3	Look for a CYCLE in the graph
No Cycle	→ SERIALIZABLE ■ Do topological sort for serial order
Has Cycle	→ NOT SERIALIZABLE ■ No equivalent serial schedule

Remember the Hierarchy: Strict $\subset$ Cascadeless $\subset$ Recoverable $\subset$ All Schedules
 Serialization ensures CORRECTNESS. Recoverability ensures SAFETY.

Textbook Reference: Fundamentals of Database Systems — Chapter 20, Pg 777, Q16, 17, 22, 23, 24